

DDoS-Protected Multi-Pod Application on Kubernetes

Information Security — Final Project

Combining DDoS Mitigation + Container Orchestration

Amantai Akunov

Problem & Motivation

The Threat

- DDoS attacks cost businesses an avg. \$40k+/hour in downtime
- Modern web apps face waves of automated bot traffic
- Slowloris, HTTP flood & layer-7 attacks exhaust resources
- Single-server architectures have no failure isolation
- Without rate limiting, one bad client = entire app down

The Need

- Defense at the network edge BEFORE traffic reaches apps
- Horizontal scalability to absorb legitimate traffic spikes
- Self-healing infrastructure that survives pod failures
- Clear separation: public gateway vs. internal services
- Reproducible deploys (config as code, not click-ops)

Our Solution

A hardened Nginx ingress gateway in front of a multi-pod K8s app

Nginx Gateway

- DDoS-hardened entry point
- Rate & connection limits
- Bot/UA filtering
- Single public endpoint

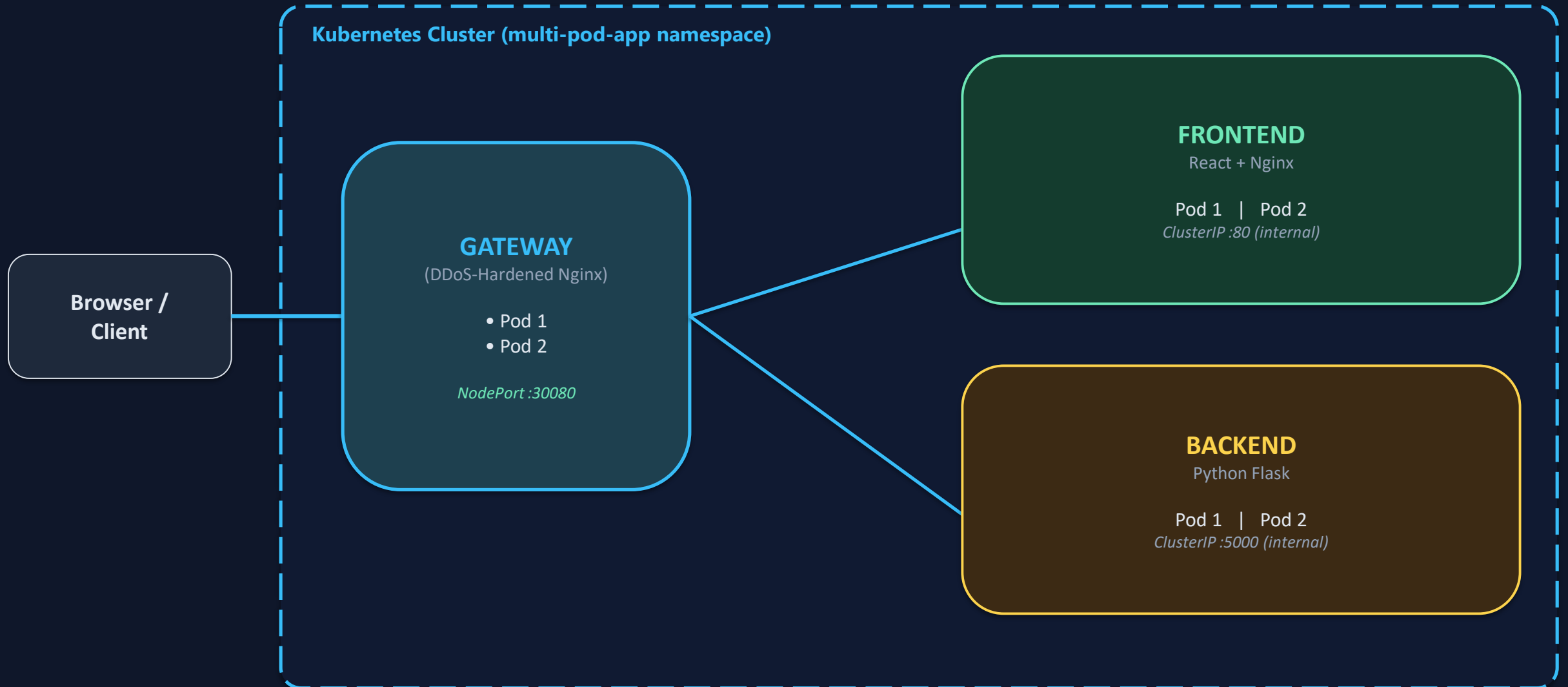
Multi-Pod App

- React frontend (SPA)
- Flask REST API backend
- 2 replicas of each
- ClusterIP — internal only

Kubernetes

- Service discovery (DNS)
- Health checks (liveness/readiness)
- Self-healing
- Horizontal scaling

System Architecture



All external traffic enters through the gateway. Frontend & backend are ClusterIP — unreachable from outside.

Key Technical Decisions

- **Nginx as gateway (not Ingress controller)**

Full control of every directive — rate limit zones, bot maps, buffer sizes — without depending on an Ingress controller's abstraction layer.

- **Frontend & backend = ClusterIP (not NodePort)**

Only the gateway is externally exposed. Internal services have zero public attack surface — bots can't bypass the gateway.

- **Return HTTP 429 instead of default 503**

429 = 'Too Many Requests' is semantically correct and tells well-behaved clients to back off. 503 implies server failure.

- **Multiple replicas per Deployment**

Two replicas of every tier (gateway, frontend, backend) — survives pod failures, distributes load, demonstrates K8s self-healing.

DDoS Protections — Gateway Layer

Layer	Nginx Directive	Effect
Rate limit	limit_req_zone — 10 r/s, burst 20	Excess → HTTP 429
Conn. limit	limit_conn_zone — 10 per IP	Excess → HTTP 429
Slowloris	client_*_timeout 10s	Closes slow connections
Buffer DoS	client_body_buffer_size 1k	Rejects oversized headers/body
Bot filter	map \$http_user_agent → bad_bot	Empty/sqlmap/nikto → HTTP 403
Methods	if (\$request_method !~ ^(GET HEAD POST)\$)	DELETE/PUT/PATCH → 444
Fingerprint	server_tokens off	Hides Nginx version
XSS/Clickjack	add_header X-Frame-Options + CSP	Browser-side hardening

Kubernetes Features Demonstrated

Namespace	<code>multi-pod-app</code>	Isolates all resources in one logical group
Deployments	<code>3 deployments × 2 replicas = 6 pods</code>	Declarative pod management with rollout strategy
Service Types	<code>NodePort (gateway) + ClusterIP (internal)</code>	Public entry, private backend
Service Discovery	<code>DNS: backend-service.multi-pod-app.svc.cluster.local</code>	Gateway resolves upstream by name, not IP
Health Probes	<code>readinessProbe + livenessProbe</code>	Auto-removes unhealthy pods from rotation
Resource Limits	<code>CPU & memory requests + limits</code>	Prevents one pod from starving the node
Load Balancing	<code>kube-proxy → endpoints</code>	Requests round-robin'd across pod replicas
Self-Healing	<code>Replica set reconciliation</code>	Killed pods get recreated within seconds

Demo — Deployment Success

./deploy.sh → builds 3 images, applies 7 manifests, waits for rollout

Screenshot 2: Successful Deployment

DDoS-Protected Multi-Pod Kubernetes Application

[4/5] Applying Kubernetes manifests...

```
namespace/multi-pod-app unchanged
deployment.apps/backend unchanged
service/backend-service unchanged
deployment.apps/frontend unchanged
service/frontend-service unchanged
deployment.apps/gateway unchanged
service/gateway-service unchanged
deployment.apps/backend restarted
deployment.apps/frontend restarted
deployment.apps/gateway restarted
All manifests applied.
```

[5/5] Waiting for pods to be ready...

```
Waiting for deployment "backend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "backend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "backend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "backend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "backend" rollout to finish: 1 old replicas are pending termination...
deployment "backend" successfully rolled out
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
deployment "frontend" successfully rolled out
deployment "gateway" successfully rolled out
```

=====

Deployment Complete!

=====

Access the application at: <http://192.168.49.2:30080>

Run the DDoS simulation test:

```
./simulate_ddos.sh http://192.168.49.2:30080
```


Demo — Running Pods & Services

kubectl -n multi-pod-app get pods + services

Screenshot 1: Kubernetes Cluster Status

DDoS-Protected Multi-Pod Kubernetes Application

```
$ kubectl -n multi-pod-app get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
backend	2/2	2	2	16m
frontend	2/2	2	2	16m
gateway	2/2	2	2	16m

```
$ kubectl -n multi-pod-app get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
backend-85689c8775-22ml6	1/1	Running	0	2m34s	10.244.0.15	minikube	<none>		<none>	
backend-85689c8775-zddq8	1/1	Running	0	2m28s	10.244.0.18	minikube	<none>		<none>	
frontend-564f5bfbb5-77v96	1/1	Running	0	2m17s	10.244.0.20	minikube	<none>		<none>	
frontend-564f5bfbb5-7pplq	1/1	Running	0	2m33s	10.244.0.16	minikube	<none>		<none>	
gateway-7f858f8445-q7mfc	1/1	Running	0	2m33s	10.244.0.17	minikube	<none>		<none>	
gateway-7f858f8445-sbccx	1/1	Running	0	2m17s	10.244.0.19	minikube	<none>		<none>	

```
$ kubectl -n multi-pod-app get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend-service	ClusterIP	10.109.163.152	<none>	5000/TCP	16m
frontend-service	ClusterIP	10.98.102.97	<none>	80/TCP	16m
gateway-service	NodePort	10.103.234.82	<none>	80:30080/TCP	16m

Demo — Gateway Routing (Frontend + Backend)

/ → frontend pods • /api/* → backend pods • all via gateway

Screenshot 3: Gateway Routes To Backend

DDoS-Protected Multi-Pod Kubernetes Application

```
$ curl -i http://192.168.49.2:30080/api/info
```

```
HTTP/1.1 200 OK
```

```
Server: nginx
```

```
Date: Sat, 09 May 2026 11:39:45 GMT
```

```
Content-Type: application/json
```

```
Content-Length: 147
```

```
Connection: keep-alive
```

```
Access-Control-Allow-Origin: *
```

```
X-Frame-Options: SAMEORIGIN
```

```
X-Content-Type-Options: nosniff
```

```
X-XSS-Protection: 1; mode=block
```

```
Referrer-Policy: no-referrer-when-downgrade
```

```
{"hostname":"backend-85689c8775-zddq8","message":"Hello from the Flask backend!","timestamp":"2026-05-09T11:39:45.896572+00:00","version":"1.0.0"}
```

Results — Rate & Connection Limiting

Test 2: Rapid-Fire (120 requests)

68

Served (HTTP 200)

52

Rate Limited (HTTP 429)

→ 43% of attack traffic blocked at the gateway

Test 3: Concurrent Connections (30)

20

Served (HTTP 200)

10

Conn. Limited (HTTP 429)

→ Exactly the configured cap of 10 concurrent per IP

Screenshot 4: DDoS Rate And Connection Limits

DDoS-Protected Multi-Pod Kubernetes Application

--- Test 1: Normal Requests (Frontend + Backend via Gateway) ---

GET / (frontend): HTTP 200 OK

GET /api/info (backend): HTTP 200 OK

GET /api/items (backend): HTTP 200 OK

Backend pod serving request: backend-85689c8775-zddq8

--- Test 2: Rapid-Fire Requests (Rate Limiting: 10r/s, burst 20) ---

Sending 120 rapid requests to the gateway...

Summary:

200 (Served): 68

429 (Rate Limited): 52

PASS: Rate limiting is active — excess requests were rejected with 429.

--- Test 3: Concurrent Connections (Limit: 10 per IP) ---

Opening 30 concurrent slow backend connections...

Summary:

200 (Served): 20

429 (Conn Limited): 10

Results — Kubernetes Load Balancing

6 sequential requests to /api/info — pods responding (round-robin):

Request 1	backend-85689c8775-22m16
Request 2	backend-85689c8775-zddq8
Request 3	backend-85689c8775-22m16
Request 4	backend-85689c8775-22m16
Request 5	backend-85689c8775-zddq8
Request 6	backend-85689c8775-zddq8

PASS — requests distributed across 2 distinct backend pods (3 + 3)

kube-proxy + Service abstraction handles routing transparently

Conclusion

A single, unified project demonstrating both topics in one architecture.

8/8

Tests passing

6

Pods (3 deployments × 2)

7

Kubernetes manifests

8+

DDoS protections

What this proves

- Both topics aren't separate concerns that they're complementary layers of a defense-in-depth design

Thank You